# Sprawozdanie z Aplikacji Mobilnych - aplikacja z przepisami na koktajle

***Wyk.*** *Tymoteusz Pawłowski 155275*
*Bartosz Kledzik 155928*

# Lab2 i lab3:

## WYMAGANIA MINIMALNE

### 1. Dwie aktywności: lista i szczegóły

**Zadanie - Aplikacja typu lista-szczegóły:**
Aplikacja składa się z dwóch aktywności:
-MainActivity.kt
-CocktailActivity.kt

W MainActivity.kt wykorzystujemy np. LazyColumn do wyświetlenia listy koktajli. Po kliknięciu na element listy, użytkownik zostaje przeniesiony do CocktailActivity za pomocą Intent.

```
val intent = Intent(contextCockTail,CocktailActivity::class.java)
intent.putExtra("cocktail_name", selectedCocktail)
contextCockTail.startActivity(intent)
```

A w CocktailActivity.kt odbieramy dane.

```
val cocktaiilName = intent.getStringExtra("cocktail_name")
```

### 2. Jetpack Compose zamiast XML

Aplikacja została napisana w kotlinie z wykorzystaniem Jetpack Compose do tworzenia UI, eliminując tradycyjne podejście oparte na XML. Zamiast tego wszystkie elementy interfejsu są tworzone za pomocą funkcji @Compose przed każdą funkcją.

### 3 i 4. Dynamiczna wielkość dla tabletów i smartphonów.

Skorzystaliśmy z ".dp" w jetpacku, który pomaga nam dynamicznie ustawiać wielkość elementów w zależności od orientacji i wielkości ekranu.

```
items(currentList) { cocktail ->
   Card(
       modifier = Modifier
           .padding(4.dp)
           .widthIn(max = 600.dp)
           .align(Alignment.CenterHorizontally)
           .clickable {
               Log.d("CocktailApp", "Kliknięto koktajl: $cocktail")
               selectedCocktail = cocktail
               val intent =
Intent(contextCockTail,CocktailActivity::class.java)
               intent.putExtra("cocktail_name", selectedCocktail)
               contextCockTail.startActivity(intent)
           },
       colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface)
   )
```

# Lab4:

#### Zadanie - Aplikacja typu lista-szczegóły: dodanie fragmentu dynamicznego

```
@Composable
fun CountdownTimer(durationSeconds: List<Int>) {
   var currentIndex by rememberSaveable { mutableStateOf(0) }
   val currentDuration = durationSeconds.getOrNull(currentIndex)

   if (currentDuration != null) {
       var timeLeft by rememberSaveable {
mutableIntStateOf(currentDuration) }
       var isRunning by rememberSaveable { mutableStateOf(false) }
       var isPaused by rememberSaveable { mutableStateOf(false) }

       LaunchedEffect(key1 = currentIndex, key2 = isRunning, key3 =
isPaused) {
           while (timeLeft > 0 && isRunning && !isPaused) {
               kotlinx.coroutines.delay(1000L)
               timeLeft--
           }
           if (timeLeft == 0 && currentIndex <
durationSeconds.size - 1) {
               isPaused = true
               currentIndex++
```

```
                timeLeft = durationSeconds[currentIndex]
            }
        }

        Column(
            modifier = Modifier.padding(top = 8.dp),
            horizontalAlignment = Alignment.Start
        ) {
            Text("⏱ $timeLeft sek", modifier = Modifier.padding(end
= 8.dp))
            Row(
                verticalAlignment = Alignment.CenterVertically,
                modifier = Modifier.padding(top = 8.dp)
            ) {

                if (timeLeft == currentDuration) {
                    Button(onClick = {
                        timeLeft = currentDuration
                        isRunning = true
                        isPaused = false
                    }) {
                        Text("Start")
                    }
                }

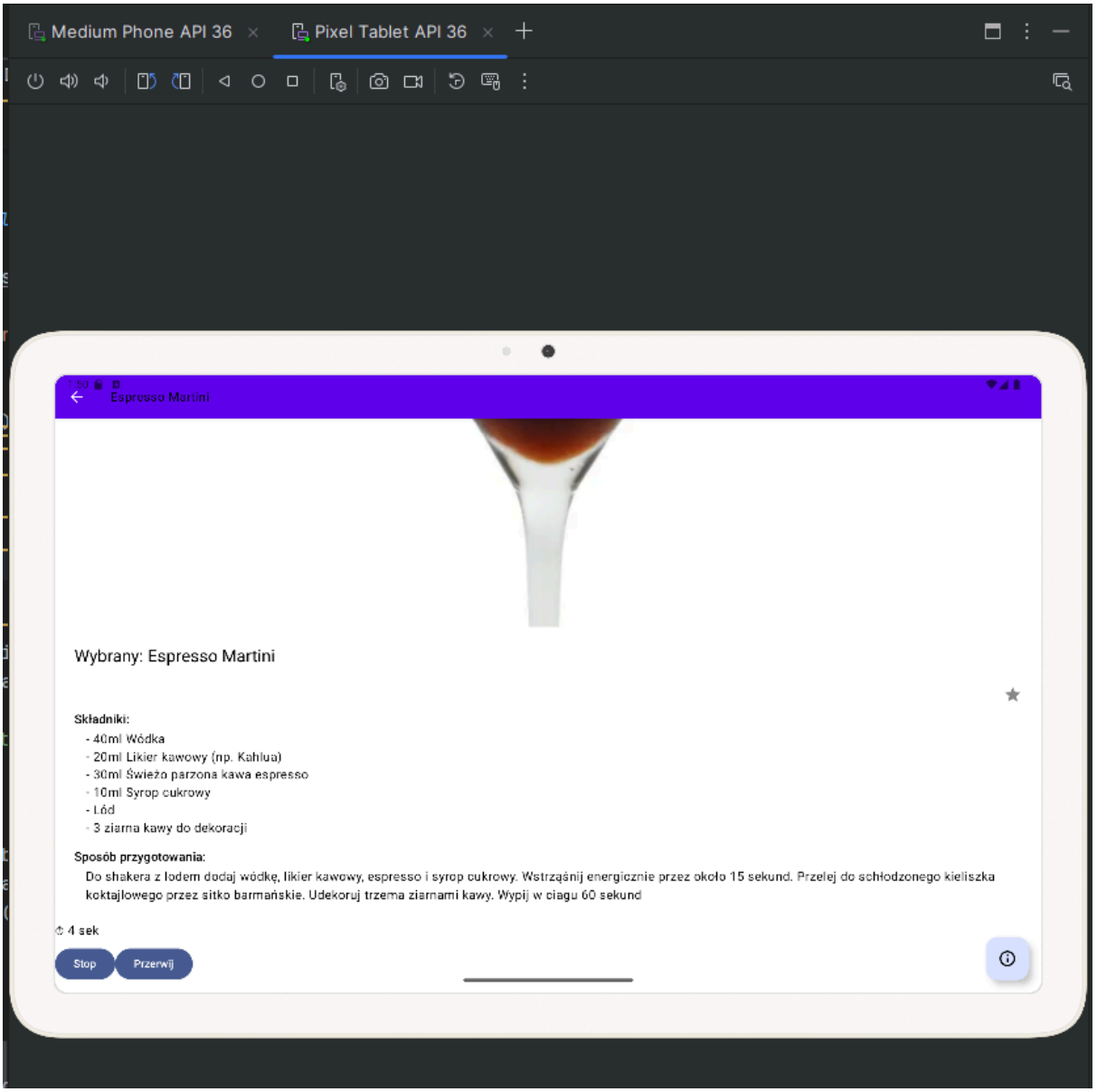

                if(timeLeft != currentDuration) {
                    Button(onClick = {
                        isRunning = false
                        timeLeft = currentDuration
                        isPaused = false
                    }) {
                        Text("Stop")
                    }
                }

                if (timeLeft in 1 until currentDuration) {
                    Button(onClick = {
                        isPaused = !isPaused
                        if (!isPaused) {
                            isRunning = true
                        }
                    }) {
                        Text(if (isPaused) "Wznów" else "Przerwij")
                    }
                }
            }
        }
```

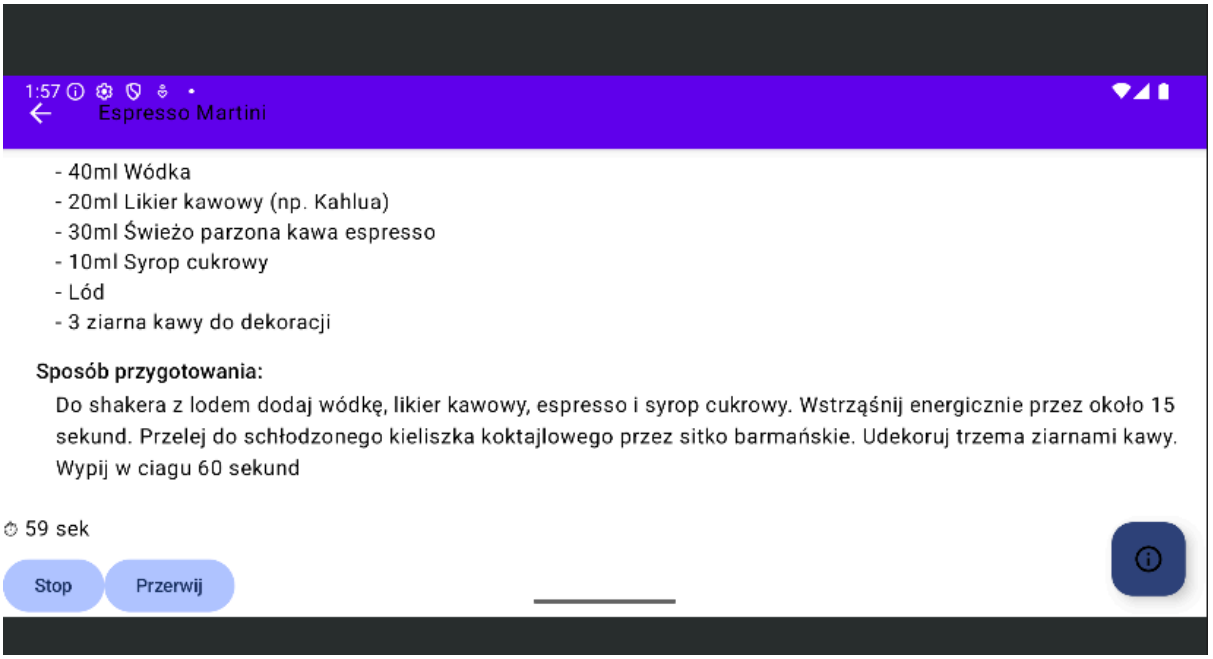
```
    }
}
```

**We fragmencie szczegółów zagnieździliśmy fragment dynamiczny minutnika. Minutnik działa na smartfonach jak i tabletach**

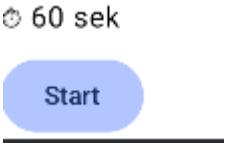




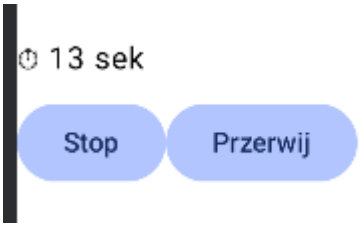

**Minutnik wyświetla czas z dokładnością do sekundy.**




**Minutnik działa poprawnie po zmianie orientacji urządzenia**
**Minutnik posiada przyciski - Start, Stop, Przerwij, Wznów**




Na początku pojawia się przycisk Start. Wciśnięcie go uruchamia mechanikę odliczania czasu.




Po naciśnięciu przycisku przerwij odliczanie się zatrzymuje oraz pojawia się w miejscu przycisku przerwij przycisk wznów.

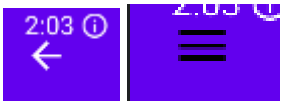



Naciśnięcie na przycisk wznów powoduje kontynuację odliczania.




Przyciśnięcie przycisku stop powoduje zresetowanie timera. I powrót do układu, w którym wyświetlony jest tylko przycisk Start odpalający timer.

**3.Umieszczenie na przyciskach ikon zamiast napisów**




Zdjęcia przedstawiają od lewej: przycisk wycofywania się z aktywności szczegółów oraz przycisk odpalający szufladę nawigacyjną.

```
navigationIcon = {
    IconButton(onClick = {
        scope.launch { drawerState.open() }
    }) {
        Icon(Icons.Filled.Menu, contentDescription = "Menu")
    }
}
```

```
drawerContent = {
    Column(
        modifier = Modifier
            .fillMaxHeight()
            .width(300.dp)
            .padding(16.dp)
    ){
    Text(
        text = "\uD83C\uDFE0 Główna Strona \uD83C\uDFE0", fontSize
= 20.sp,
        modifier = Modifier

            .fillMaxWidth().padding(16.dp)
            .clickable {
                scope.launch { drawerState.close() }
                navController.navigate("cocktailList") {
popUpTo("cocktailList") }
            }
            .padding(top = 50.dp)
    )
    Text(
        text = "\u2B50 Ulubione \u2B50", fontSize = 20.sp,
        modifier = Modifier
            .fillMaxWidth().padding(10.dp)
            .clickable {
                scope.launch { drawerState.close() }
                navController.navigate("favorites") {
popUpTo("cocktailList") }
            }
            .padding(top = 30.dp)
    )
}
```

```
}
```

Kod odpowiedzialny za obsługę szuflady nawigacyjnej powyżej.

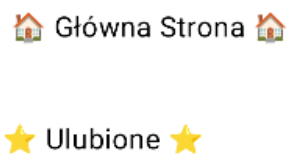
```
navigationIcon = {
   IconButton(onClick = {
activity?.onBackPressedDispatcher?.onBackPressed() }) {
       Icon(Icons.Default.ArrowBack, contentDescription = "Wstecz")
   }
}
```

Kod odpowiedzialny za obsługę strzałki wstecz powodującej powrót do poprzedniej aktywności.

**4.Dodatkowe funkcje/dodatkowe przyciski**



Aby przejść do zakładki ulubione należy wejść w szufladę nawigacyjną i nacisnąć na przycisk Ulubione






```
drawerContent = {
    Column(
        modifier = Modifier
            .fillMaxHeight()
            .width(300.dp)
            .padding(16.dp)
    ){
        Text(
            text = "\uD83C\uDFE0 Główna Strona \uD83C\uDFE0",
fontSize = 20.sp,
            modifier = Modifier

                .fillMaxWidth().padding(16.dp)
                .clickable {
                    scope.launch { drawerState.close() }
                    navController.navigate("cocktailList") {
popUpTo("cocktailList") }
                }
                .padding(top = 50.dp)
        )
        Text(
            text = "\u2B50 Ulubione \u2B50", fontSize = 20.sp,
            modifier = Modifier
                .fillMaxWidth().padding(10.dp)
                .clickable {
                    scope.launch { drawerState.close() }
                    navController.navigate("favorites") {
popUpTo("cocktailList") }
                }
                .padding(top = 30.dp)
        )
    }
}
```

Kod odpowiedzialny za obsługę zawartości szuflady nawigacyjnej. Po naciśnięciu przycisku Główna Strona następuje przekierowanie do funkcji obsługującej listę wszystkich koktajli, a po naciśnięciu na przycisk Ulubione do funkcji obsługującej listę tych ulubionych.

Przycisk obsługujący dodawanie napoju do ulubionych:




```
@Composable
fun FavoriteStar(cocktailName: String) {
    val context = LocalContext.current
    var favorites by remember {
mutableStateOf(loadFavorites(context).toMutableSet()) }

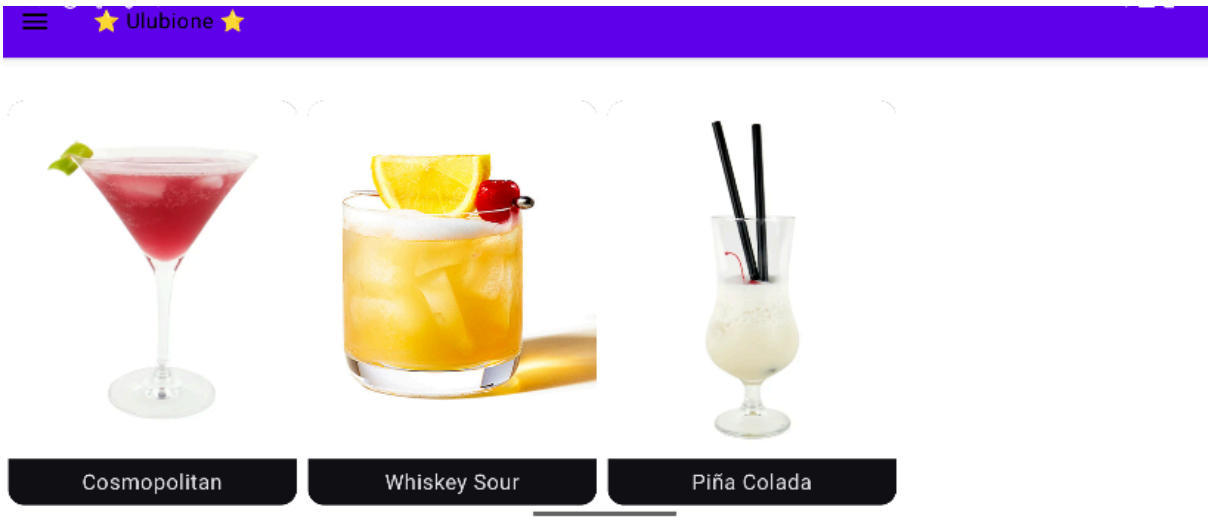
    val currentFavorites = rememberUpdatedState(favorites)

    val isFavorite = currentFavorites.value.contains(cocktailName)
    val HoneyYellow = Color(0xFFFFC107)
    Icon(
        imageVector = Icons.Default.Star,
        contentDescription = null,
        tint = if (isFavorite) HoneyYellow else Color.Gray,
        modifier = Modifier.clickable {
            val updatedFavorites =
currentFavorites.value.toMutableSet()
            if (isFavorite) {
                updatedFavorites.remove(cocktailName)
            } else {
                updatedFavorites.add(cocktailName)
            }
            saveFavorites(context, updatedFavorites)

            favorites = updatedFavorites
        }
    )
}
```

Kod obsługujący mechanikę dodawania napoju do ulubionych

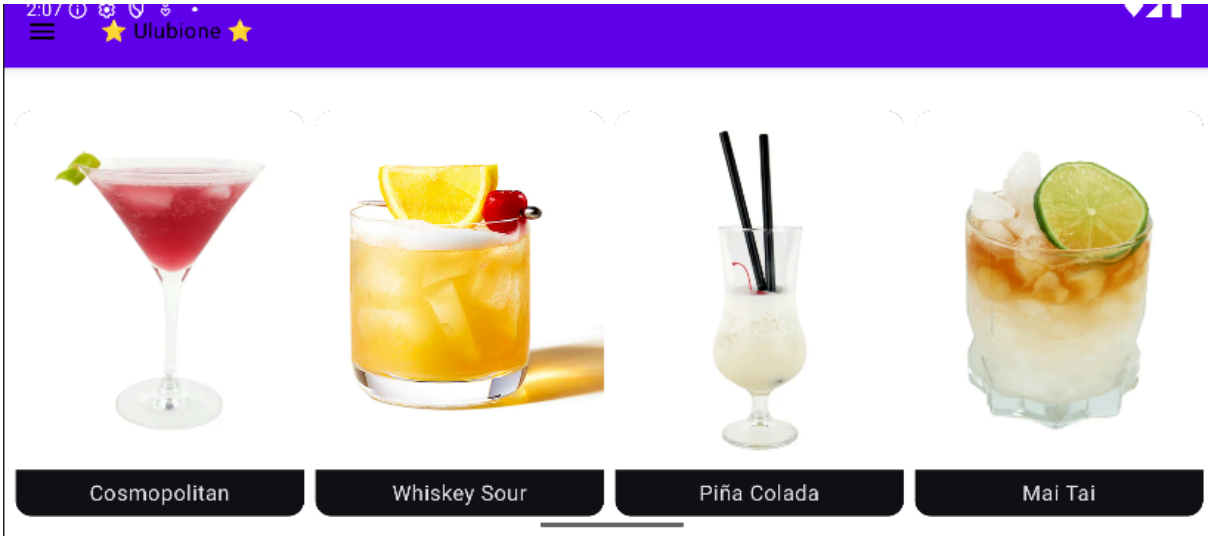
Stan przed dodaniem napoju do ulubionych:




Po naciśnięciu na przycisk gwiazdki następuje dodanie drinka do zakładki ulubione. Aby usunąć dany drink z tej zakładki wystarczy po raz kolejny nacisnąć na ten przycisk powróci on wtedy do stanu domyślnego i drink zostanie usunięty z ulubionych.




Stan po dodaniu przycisku do ulubionych:



# Lab5,6 i 7

### Zadanie - Elementy biblioteki wsparcia wzornictwa (zadanie potrójne)

1.

```
LazyVerticalGrid(
    columns = GridCells.Adaptive(minSize = 200.dp),
    modifier = Modifier
        .weight(1f)
        .padding(end = 4.dp, top = 20.dp)
) {
    items(currentList) { cocktail ->
        Card(
            modifier = Modifier
                .padding(4.dp)
                .widthIn(max = 600.dp)
                .align(Alignment.CenterHorizontally)
                .clickable {
                    Log.d("CocktailApp", "Kliknięto koktajl:
$cocktail")
                    selectedCocktail = cocktail
                    val intent =
Intent(contextCockTail,CocktailActivity::class.java)
                    intent.putExtra("cocktail_name",
selectedCocktail)
                    contextCockTail.startActivity(intent)
                },
            colors = CardDefaults.cardColors(containerColor =
MaterialTheme.colorScheme.surface)
        )
```

MainActivity.kt

```
LazyColumn(
    state = scrollState,
    modifier = Modifier
        .fillMaxSize()
        .padding(innerPadding)
)
```

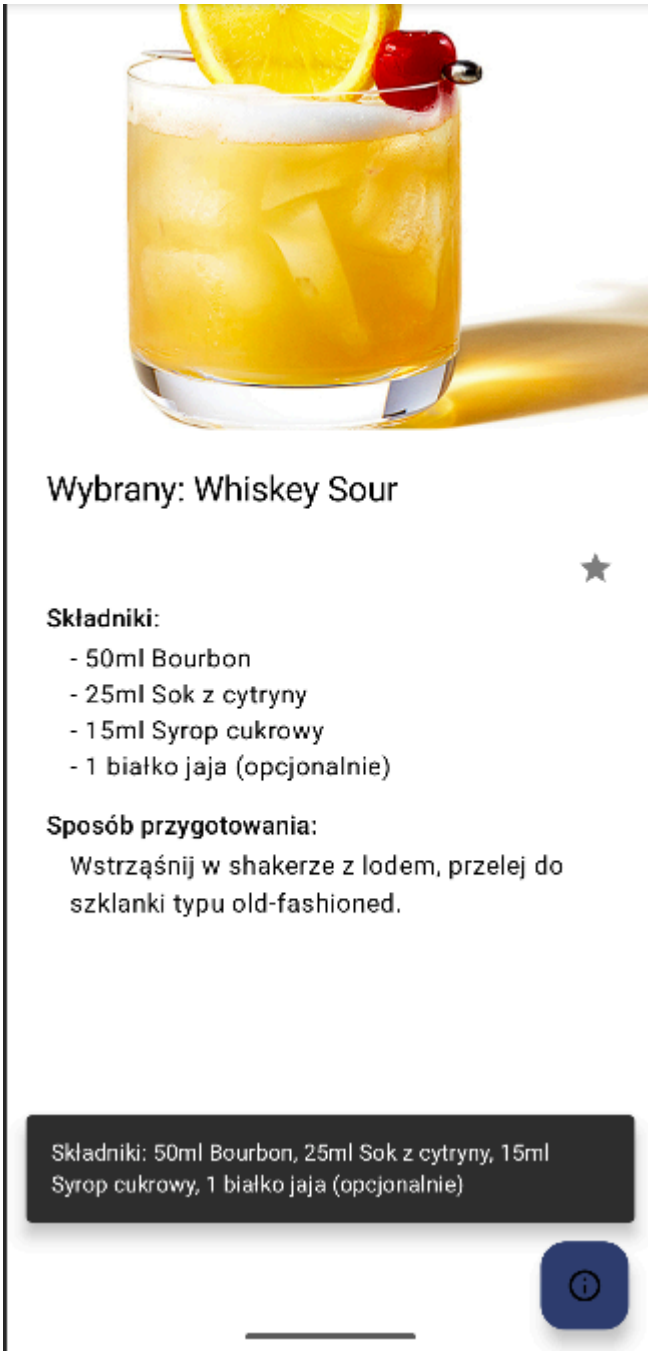
**CocktailActivity.kt**

W jetpackCompose odpowiednikiem CardView i RecyclerView jest np LazyVerticalGrid()/LazyColumn() oraz Card().

**2. Przycisk FAB**




**Ekran przed wciśnięciem FAB**




**Ekran po wciśnięciu**

Przycisk FAB wyświetla aktualne składniki potrzebne do przygotowania koktajlu jako powiadomienie.

```
floatingActionButton = {
    FloatingActionButton(
        onClick = {
            val (ingredients, _) =
cocktailsDetails[selectedCocktail] ?: Pair(emptyList(), "")
            val message = if (ingredients.isNotEmpty()) {
                "Składniki: ${ingredients.joinToString(", ")}"
            } else {
                "Brak danych dla $selectedCocktail"
            }
            coroutineScope.launch {
                snackbarHostState.showSnackbar(message)
            }
        }
    ) {
        Icon(Icons.Outlined.Info, contentDescription = "Pokaż
składniki")
    }
})
```

**3. Motywy:**

W aplikacji wyciągamy ustawienia dot. motywu ciemnego lub jasnego z androida i stawiamy odpowiednie kolory.

```
private val DarkColorScheme = darkColorScheme(
primary = Purple40,
    secondary = PurpleGrey40,
    tertiary = Pink40,
    surface= Surface,
    onPrimary = Color.White,
    onSurface = Color.White,
)

private val LightColorScheme = lightColorScheme(
    primary = Purple40,
    secondary = PurpleGrey40,
    tertiary = Pink40,
    surface= Surface,
    onPrimary = Color.Black,
    onSurface = Color.Black,
)

@Composable
fun MobileCocktailsTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
```

```
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >=
Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else
dynamicLightColorScheme(context)
        }

        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }

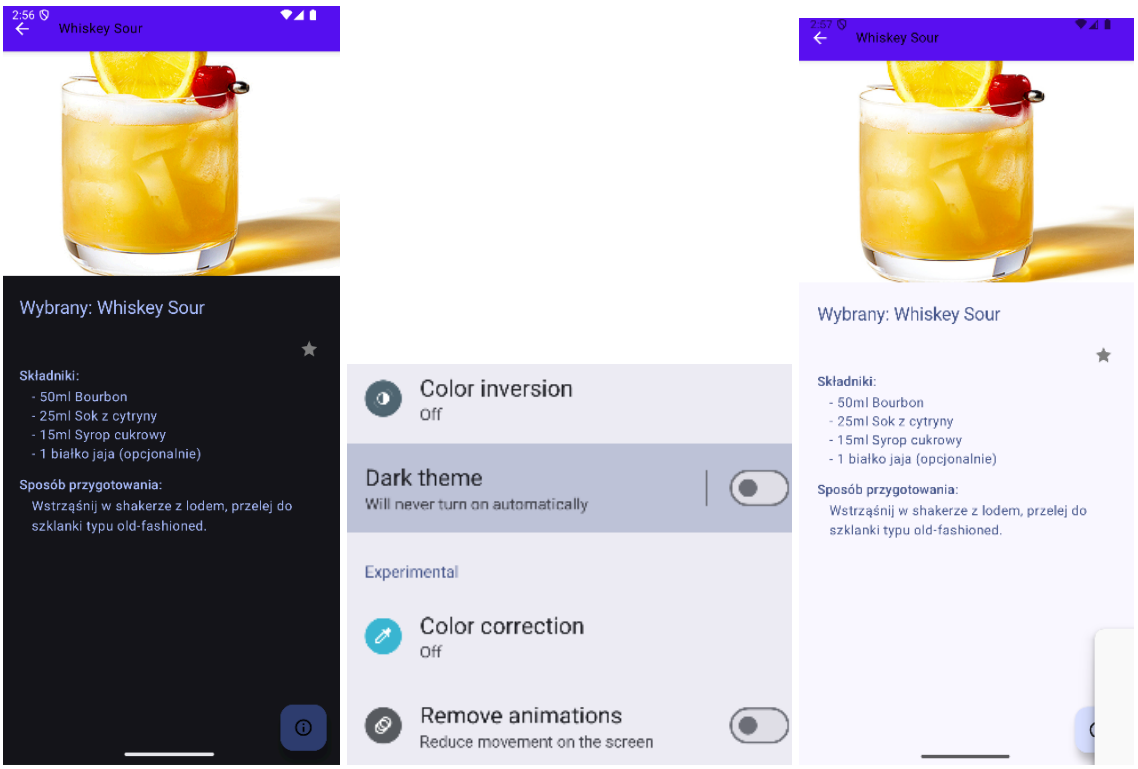
    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

*plik theme.kt*

```
item {
    Text(
        text = "Wybrany: $cocktailName",
        style = MaterialTheme.typography.titleLarge,
        color = MaterialTheme.colorScheme.primary, //zmiana motywu
        modifier = Modifier
            .padding(25.dp)
    )
```

*Przykład z kodu gdzie ustawiamy kolor tekstu na aktualny wybrany motyw*

(W filmiku była przestarzała wersja obsługi motywów, w załączonych zdjęciach widać większą różnice w motywie ciemnym i jasnym)




*Zmiana motywu w systemie android dynamicznie zmienia nam kolor tekstu i tła.*

**4. W aplikacji należy korzystać z fragmentów (w przypadku Jetpack Compose nie wymagane):** Stosowaliśmy Jetpack Compose

**5. Aplikacja ma działać poprawnie przy zmianie orientacji urządzenia**

W naszej aplikacji, aby poprawnie obsłużyć zmiany orientacji ekranu (np. przejście z widoku pionowego do poziomego), wykorzystaliśmy funkcję rememberSaveable w Jetpack Compose. Funkcja ta zapewnia przechowywanie stanu pomiędzy zmianami konfiguracji, dzięki czemu wartości zmiennych, takich jak wybrany koktajl czy aktualny czas na zegarze są automatycznie zapisywane i przywracane.

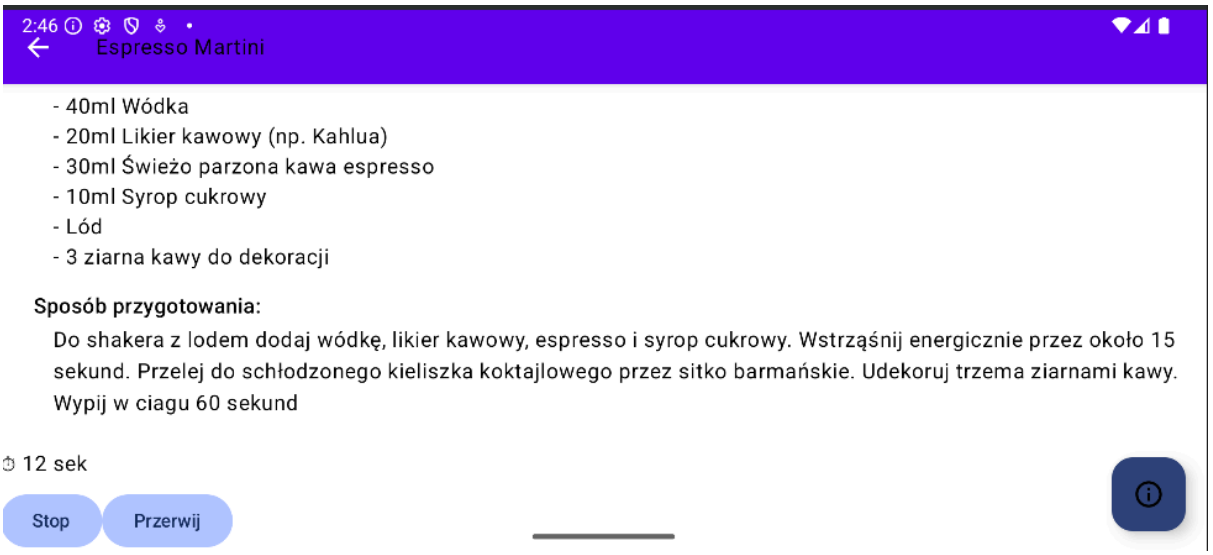
```
fun CountdownTimer(durationSeconds: List<Int>) {
    var currentIndex by rememberSaveable { mutableStateOf(0) }
    val currentDuration = durationSeconds.getOrNull(currentIndex)

    if (currentDuration != null) {
        var timeLeft by rememberSaveable {
mutableIntStateOf(currentDuration) }
        var isRunning by rememberSaveable { mutableStateOf(false) }
        var isPaused by rememberSaveable { mutableStateOf(false) }
```

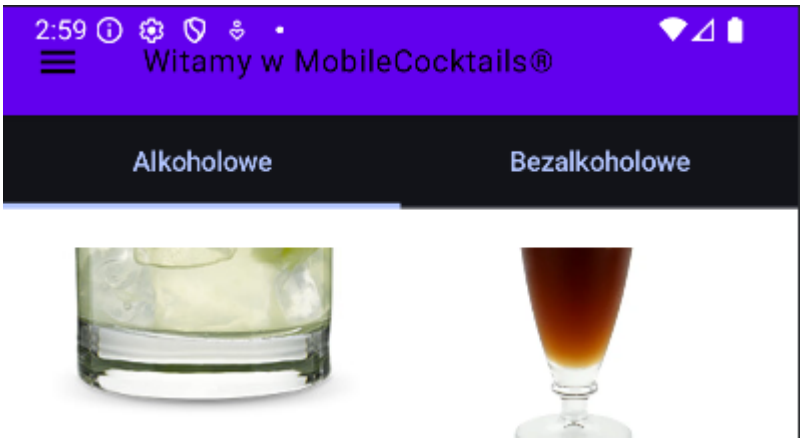
Wersja w pionie:




## Wersja w poziomie:



**6. Każda aktywność ma mieć pasek aplikacji w postaci paska narzędzi**




Powyżej aktywność główna




Powyżej aktywność szczegółów

```
topBar = {
    androidx.compose.material.TopAppBar(
        title = { Text("Witamy w MobileCocktails\u00AE") },
        navigationIcon = {
            IconButton(onClick = {
                scope.launch { drawerState.open() }
            }) {
                Icon(Icons.Filled.Menu, contentDescription = "Menu")
            }
        }
    )
},
```

topBar wbudowany w scaffolda pozwala nam tworzyć pasek aplikacji który podstawowo jest “przyklejony” na górze i przesuwa się wraz z scrollowaniem.

**7.Ekran szczegółów ma być przewijany w pionie razem z paskiem aplikacji**




Wybrany drink przed przewinięciem w dół

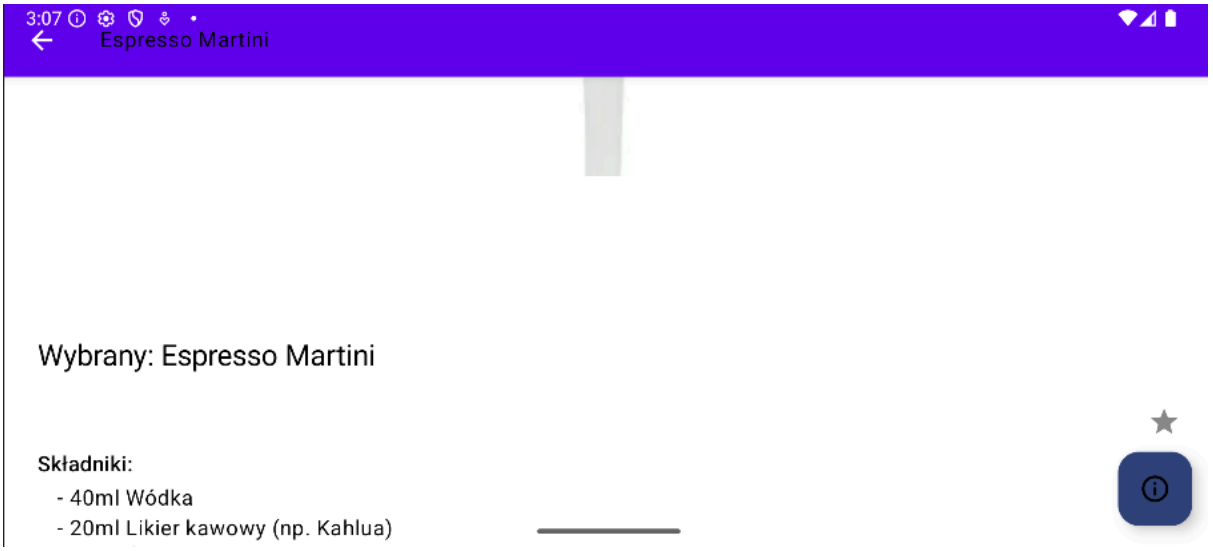



Wybrany drink po przewinięciu w dół. Jak widać pasek aplikacji zostaje u góry ekranu. Mechanika utrzymywania paska aplikacji u góry ekranu została opisana w poprzednim punkcie.

**8.Na ekranie szczegółów obrazek ma się pojawić na pasku aplikacji, ale ma się razem z nim zwijać:**




Jak widzimy na zdjęciu na początku obrazek widnieje cały na górze ekranu szczegółów.




Wraz z przewijaniem w dół ekranu obrazek zaczyna się chować.

```
item {
    val scrollOffset =
minOf(scrollState.firstVisibleItemScrollOffset,
with(LocalDensity.current) { maxHeight.dp.toPx() }.toInt())

    Image(
        painter = painterResource(id = if (imageResId != 0)
imageResId else R.drawable.trollface),
        contentDescription = "$selectedCocktail Image",
        contentScale = ContentScale.Crop,
        modifier = Modifier
            .height(maxHeight.dp)
            .graphicsLayer {
                translationY = -scrollOffset.toFloat()
            }
            .fillMaxWidth()
    )
}
```

ustawienie:

translationY = -scrollOffset.toFloat()

sprawia, że wraz ze scrollowaniem w dół ekranu obraz zaczyna stopniowo chować się do góry aż do całkowitego zniknięcia.

**9.Przechodzenie pomiędzy kartami za pomocą gestu przeciągnięcia:**

NavController oraz Navigation Component tworzy “stos ekranów” przez co gest cofnięcia wbudowany w androida wie gdzie ma cofnąć użytkownika.

**10. Szuflada nawigacyjna**

Opisano w Lab4 podpunkt 4.

# Lab8

## Zadanie - Animacje

Animacja wyświetla 3 losowe drinki przepływające z prawej strony ekranu do lewej w trakcie ładowania aplikacji. (Najlepiej widoczna na wysłanym filmiku)





```
@Composable
fun LoadingScreen(navController: NavController)
{
    val context = LocalContext.current

    val allCocktails = remember {
        listOf("Cosmopolitan", "Whiskey Sour", "Piña Colada", "Mai
Tai","Daiquiri", "Manhattan","Mojito", "Gin Fizz", "Caipirinha",
"Long Island Iced Tea","Negroni", "Bloody Mary", "Tequila
Sunrise","Espresso Martini","★Virgin Mojito★", "★Shirley
Temple★", "★Lemonade★")
    }

    val uniqueRandomList =
(0..allCocktails.size-1).shuffled().take(3)
    var vCounter by remember { mutableStateOf(0) }
    val imageResIds = remember {
        allCocktails.mapNotNull { cocktail ->
```

```
            val imageName = cocktail
                .lowercase()
                .replace(" ", "")
                .replace("ñ", "n")
                .replace("[^a-z0-9_]".toRegex(), "")
                .replace("★", "")
            val resId = context.resources.getIdentifier(imageName,
"drawable", context.packageName)
            if (resId != 0) resId else null
        }
    }
    var currentImageIndex by remember {
mutableStateOf(uniqueRandomList[0])}
    val imageViewRef = remember { mutableStateOf<ImageView?>(null) }

    LaunchedEffect(currentImageIndex) {
        val imageView = imageViewRef.value
        imageView?.let {

            it.translationX = 1000f
            it.setImageResource(imageResIds[currentImageIndex])


            val animator = ObjectAnimator.ofFloat(it,
"translationX", 1000f, -1000f)
            animator.duration = 1500
            animator.start()

            delay(1500)

            if (vCounter < 2) {
                vCounter += 1

                currentImageIndex = uniqueRandomList[vCounter]
            } else {
                navController.navigate("splashScreen") {
                    popUpTo("loadingScreen") { inclusive = true }
                }
            }
        }
    }
```

Kod obsługujący animację wykorzystując ObjectAnimator
wykorzystanie ObjectAnimator

```
val animator = ObjectAnimator.ofFloat(it, "translationX", 1000f,
-1000f)
            animator.duration = 1500
            animator.start()
```